

Seminar Paper on APL Ethernet

1st Raihan Uddin
Electronic Engineering
Student ID: 2210018
Hochschule Hamm-Lippstadt
Lippstadt, Germany
raihan.uddin@stud.hshl.de

2nd Bodrul Amin Murad
Electronic Engineering
Student ID: 2210039
Hochschule Hamm-Lippstadt
Lippstadt, Germany
bodrul-amin.murad@stud.hshl.de

3rd Ismail Hossain
Electronic Engineering
Student ID: 2210015
Hochschule Hamm-Lippstadt
Lippstadt, Germany
ismail.hossain@stud.hshl.de

Abstract—
Index Terms—

I. INTRODUCTION

The Cross Traffic Management project deals with the complexity of efficient routing of autonomous vehicles through a four-way intersection. Conventional traffic control methods lack the adaptability and responsiveness needed to manage real-time vehicle flows for autonomous vehicles. To avert these limitations, our implemented system, here, employs a max dynamic weight scheduling algorithm— dynamically assigning priority to each traffic direction based on real-time queue lengths, vehicle arrival rates, and safety constraints.

- (a) **Core Logic in C:** The foundational behavior of the intersection controller was implemented in C. This stage established the data structures and algorithms needed to model vehicle queues and calculate dynamic weights for each direction.
 - (b) **FreeRTOS Integration:** To support real-time execution and concurrent processing of sensor data, queue management, and scheduling decisions, the C implementation was ported to FreeRTOS. This ensured deterministic task scheduling and inter-task communication under realistic load conditions.
 - (c) **UPPAAL Verification:** Using UPPAAL, we formally modeled the system's timing behavior and verified critical properties such as deadlock freedom, bounded waiting time, and correct weight recalculation. This step provided mathematical guarantees of system correctness before hardware realization.
 - (d) **VHDL Hardware Design:** Finally, the scheduling logic was translated into VHDL and simulated in ModelSim. This hardware-oriented implementation demonstrates the feasibility of deploying the controller on FPGA or ASIC platforms for ultra-low-latency intersection management.
- To achieve the objectives of efficient cross-traffic management, our work is structured around the following research questions:
- (a) **Queuing Model:** How can we accurately model intersection traffic for autonomous vehicles using queuing theory?
 - (b) **Optimal Scenario:** What scheduling scenarios and weight functions best minimize congestion and waiting times?

- (c) **Simulation Framework:** How can we simulate the intersection controller—first in C and then under FreeRTOS—and which parameters (e.g., arrival rates, service times, task priorities) are critical to assess performance?
- (d) **Hardware Realization:** Which components of the system are best suited for hardware implementation, and how can VHDL simulation validate their real-time behavior?

By combining software development, real-time operating systems, formal verification, and digital circuit design, this project delivers a scalable and robust traffic control solution suitable for smart intersections in autonomous vehicle networks.

II. SCHEDULING ALGORITHM

In our centralized traffic management system, we have used dynamic-weight scheduling algorithm to efficiently route autonomous vehicles at the junction. For algorithm design, we sought online help that is browsing for critical steps in ChatGpt. According to suggested guidelines, we designed our algorithm as follows:

A. Overview and Objectives

The algorithm is meant to achieve:

- (a) Dynamic prioritization of competing traffic directions based on real-time metrics.
- (b) Fairness (no direction is starved) and throughput optimization (minimize overall waiting time).
- (c) Safety constraints, e.g., minimum green times, no conflicting movements.

B. System Model and Metrics

The input and state variables of the scheduler are:

- (a) Queue length Q_i for each direction $i \in \{N, S, E, W\}$
- (b) Arrival rate λ_i (vehicles/sec) estimated over a sliding window
- (c) Waiting time W_i of the lead vehicle in the queue
- (d) Safety timer $T_{\min}$ ensuring a green phase lasts at least some minimum

C. Weight Computation

Present the weight function that combines these metrics into a single score per direction: $w_i = \alpha Q_i + \beta \lambda_i + \gamma W_i$ where, α , β and γ are tunable coefficients that reflect the relative importance of queue length, flow rate and waiting time.

D. Algorithm Steps

The loop of the scheduler will work like follows:

- 1) Sense and Update
 - (a) Read current queue lengths Q_i and arrival rates of λ_i .
 - (b) Update waiting times W_i for vehicles in the queue.
- 2) Compute weights
 - (a) For each direction i , we need to compute w_i .
- 3) Select Max-Weight Direction
 - (a) We have to select

$$d = \arg \max_i (w_i)$$
 - (b) Enforce safety constraints, if the previously green directions has not yet met $T_{\min}$, delay switching.
- 4) Grant Green phase
 - (a) Activate green for direction d .
 - (b) Start or reset the green-phase timer T_d
- 5) Repeat
 - (a) After either a fixed epoch $T_{\max}$ or when w_d falls below the next-highest weight by a threshold, return to the initial point.
- 6) Properties and Tuning
 - (a) **Fairness:** Because every direction's weight eventually grows (via W_i), starvation is prevented.
 - (b) **Responsiveness:** By tuning α , β and γ , the system can emphasize clearing long queues or reacting to sudden surges.
 - (c) **Safety Margins:** $T_{\min}$ avoids rapid toggling.

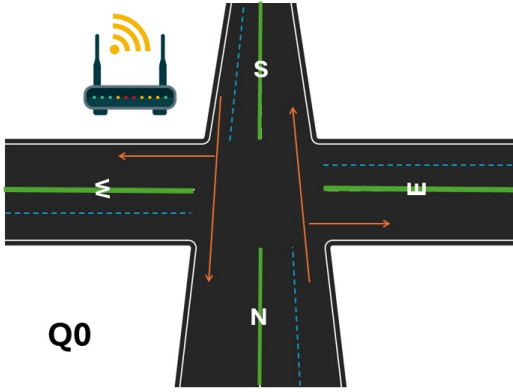


Fig. 1. Q0

III. SysML DIAGRAM

We have used different SysML diagrams to visually represent the system's structure and behavior.

A. Activity Diagram

This diagram models the runtime behavior of a single vehicle as it interacts with the intersection controller:

- 1) Receive arrival + turn intent

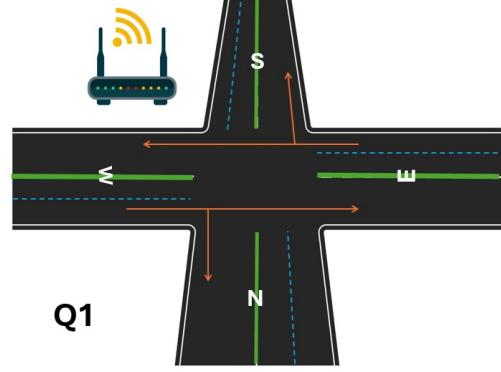


Fig. 2. Q1

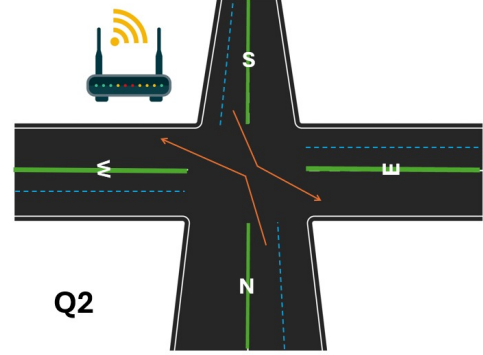


Fig. 3. Q2



Fig. 4. Q3

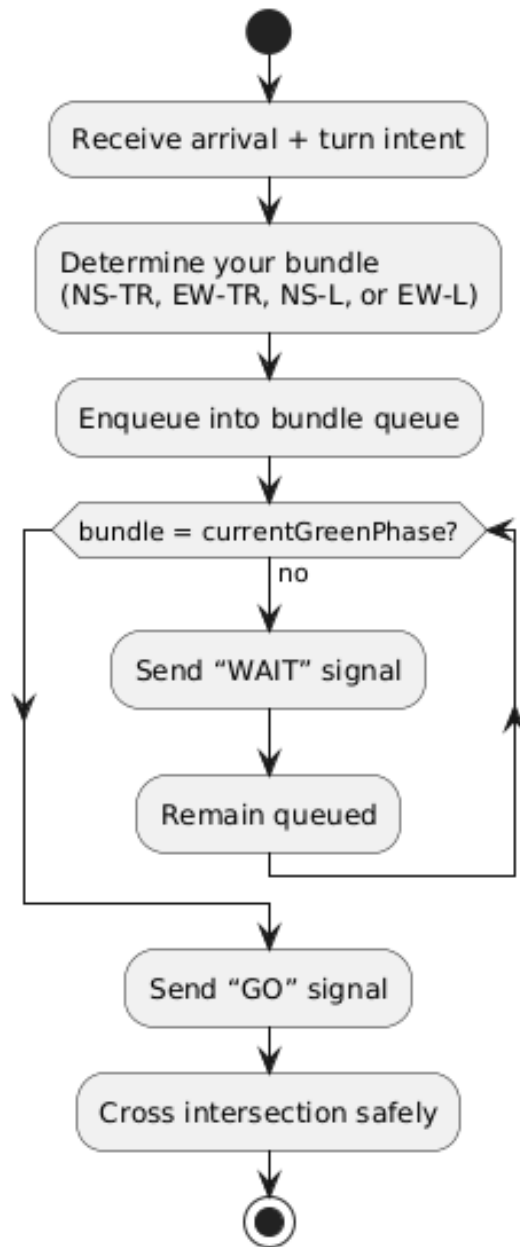


Fig. 5. Activity diagram

- (a) A vehicle arrives at the stop line and communicates its intended movement (straight, left, or right).
- 2) Determine bundle
 - (a) Based on direction (North South or East West) and turn intent (through or left), the vehicle is assigned to one of four “bundles”: NS-TR, EW-TR, NS-L, or EW-L.
- 3) Enqueue into bundle queue
 - (a) The controller places the request in the corresponding queue for later scheduling.
- 4) Check against current green phase
 - (a) If the vehicle bundle matches the currently active

(green) phase, it bypasses waiting; otherwise:

- (a) Send “WAIT” signal
- (b) Remain queued until the green phase switches back.
- 5) Send “GO” signal
 - (a) When its bundle is selected by the scheduling algorithm, the controller grants green.
- 6) Cross intersection safely
 - (a) The vehicle proceeds through the junction under the granted green signal.

B. Requirement Diagram

This UML requirements diagram organizes and decomposes the system-level requirement “Intersection Control” into five constituent requirements:

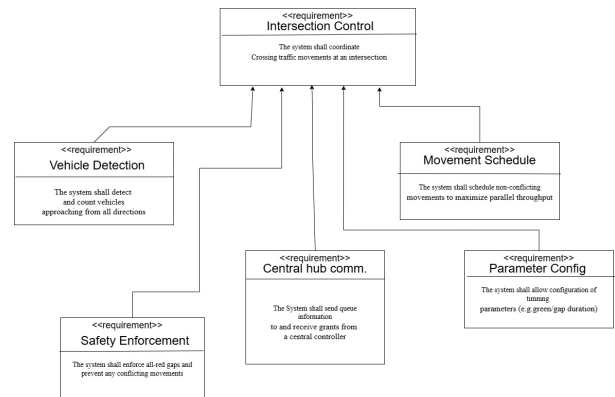


Fig. 6. Requirement diagram

- 1) Vehicle Detection
 - (a) “The system shall detect and count vehicles approaching from all directions”.
- 2) Movement Schedule
 - (a) “The system shall schedule non-conflicting movements to maximize parallel throughput”.
- 3) Central Hub Communication
 - (a) “The system shall send queue information to and receive grants from a central controller”.
- 4) Parameter Configuration
 - (a) “The system shall allow configuration of timing parameters (e.g., green/gap duration)”.
- 5) Safety Enforcement
 - (a) “The system shall enforce all-red gaps and prevent any conflicting movements”.

C. Use Case Diagram

This diagram captures the key interactions between two actors and the Infrastructure Intersection Management System:

- 1) **Actor:** Car
 - (a) Enter Queue
 - (b) Enter/Request to Enter Junction

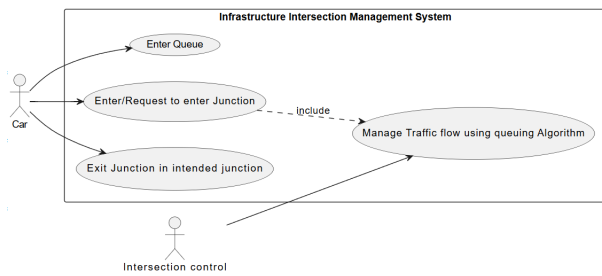


Fig. 7. Requirement diagram

(a) Includes the “Manage Traffic Flow using Queuing Algorithm” use case.

(c) Exit Junction

2) **Actor:** Intersection Control

(a) Invokes “Manage Traffic Flow using Queuing Algorithm” (representing the central controller or coordinating entity that runs the scheduling logic).